

Developer (LLM) Take-Home Assignment

1. Task

Build a question-and-answer chatbot web application (locally) that answers financial questions about U.S. public companies. Every answer must be grounded in the data we provide: structured financials held in a SQL database, and qualitative text extracted from 10-K annual filings held in a vector database. The application must include user authentication (will be later used in a live coding session).

The defining requirement is no hallucination. When the data needed to answer a question is not available, the assistant must say so clearly rather than invent figures or explanations.

2. What you will build

Area	Requirement
Chat application	A web UI plus a backend API where a signed-in user can ask financial questions and get grounded answers.
Retrieval	Answer using a SQL database (structured financials) and a vector database (10-K text), choosing the right source per question.
Authentication	Users sign in to use the app. Keep it clean and easy to extend; we may build on it during the live session.
Local infra	Bring up the SQL and vector databases locally with Docker Compose and load the data we provide.

3. Questions the chatbot must able to answer

As a baseline, your chatbot must correctly answer the following three questions.

Q1: สรุป net income ปี 2022-2025 ของบริษัท Apple

Q2: เปรียบเทียบโครงสร้างรายได้และกลยุทธ์ทางธุรกิจของ Google และ Facebook เพื่อประเมินจุดแข็งและจุดอ่อนในตลาด ปี 2025

Q3: จากรายได้ของบริษัท Microsoft, Apple, Google, Facebook ในปี 2024-2025 บริษัทใดที่มีอัตราการเติบโตสูงสุด และอะไรเป็นปัจจัยหลัก

4. Provided Data

You receive a folder with the raw source data.

Item	Description
10k_filings/	Four FY2025 10-K filings as PDFs: Alphabet, Amazon, Apple, Meta. These are the raw source documents for the qualitative and strategy content.
data/financial_data.sql	A PostgreSQL dump (schema plus COPY data). Income-statement data for ~48 U.S. companies, 2022-2025 (192 rows). Columns: company, ticker, sector, year, revenue, gross_profit, operating_income, net_income (all in USD).
data/pinecone_vectors.jsonl.gz	The “10-K filings” text where it has been already chunked and embedded, ready to load straight into a local Pinecone index (the fast path). See section 6.
OpenAI API key	Provided separately. Use it for the LLM and for query-time embeddings.

Data coverage (read this carefully)

Structured (SQL) covers ~48 companies for 2022-2025. The vector store covers only the four FY2025 10-K filings (Alphabet, Amazon, Apple, Meta). So a company can have financial numbers but no filing text. For example, there are SQL figures for Microsoft, but no Microsoft 10-K to ground a “why” answer.

5. Tech stack

- **Backend:** FastAPI (Python), Express.js or NestJS are preferred.
- **Frontend:** Your choice.
- **LLM and embeddings:** OpenAI (key provided). Candidates should use an agent framework such as LangChain or LangGraph.
- **Databases:** PostgreSQL for the structured data and Pinecone for the vectors, both running locally (see section 6).

6. Data infrastructure: run it locally with Docker Compose

Stand up Postgres and a local Pinecone yourself with Docker Compose. (Pinecone publishes a local image, pinecone-local, that the SDK can target by overriding the host.) Designing this local stack is part of the assignment.

Load the SQL

data/financial_data.sql is a standard pg_dump-style file: a DDL prelude followed by a COPY FROM stdin block of tab-separated rows. Restore it into your Postgres with psql, or a small loader that runs the DDL and then streams the COPY block.

Load the vectors (two options)

1. **Option A, use the fixture.** Upsert data/pinecone_vectors.jsonl.gz directly. Each line is one JSON object with id, namespace, values (the embedding), and metadata. We have

extracted text per page from each PDF (PyPDF), split it with a recursive character splitter (chunk_size = 1000, overlap = 200), embedded each chunk with OpenAI text-embedding-3-small at dimensions = 512, and upsert into a cosine index, storing the chunk text, page, and source in each vector's metadata.

2. **Option B, re-ingest.** If you want to control chunk size or embedding dimension, ignore the fixture and build the index from the raw PDFs in 10k_filings/

7. Authentication

Implement user authentication so users sign in to use the app. We may use your authentication for the test in the live coding session, so keep the auth and data-access layers clean and easy to extend.

8. Deliverables

1. A Git repository containing your source code, a docker-compose for the local data stack, and any seed or loader scripts.
2. A README with setup steps: how to bring up the databases, how to load the data, and how to run the app.
3. The running application answers Q1 to Q3 above.

9. Note

Use AI assistants, but own your code

You may use AI coding tools. But you must fully understand, and be able to explain and extend, every part of your codebase. In the live session we will ask you to walk through your design and then implement surprise extensions on top of it.

Budget Limit

The provided OpenAI API key is capped at US\$10 which we think it's plenty for this task. You are also free to use another AI provider you already have, if you prefer; it is your choice.